

Programación

PARTE 1: Herencia, Interfaces, Composición, Agregación

Ejercicio Base. Realiza un programa base en el método main de un nuevo proyecto para el examen. Este programa debe de tener un menú repetitivo como en exámenes previos (salir con el 0).

Descripción del modelo a programar

Vamos a realizar un prototipo para gestionar las reparaciones mecánicas a realizar en un taller de una famosa marca de vehículos. Un taller tiene un CIF y una descripción. En nuestro taller se reparan motocicletas, turismos y furgones de hasta 3500Kg (vehículos industriales ligeros). Todos los vehículos poseen una **matrícula**, el **año de fabricación** y la **descripción del modelo**. Por ejemplo "Seat León 1.4 TSI", "Renault Transit 1.6dci" o "Yamaha 125 City". Los vehículos pertenecen a nuestros clientes, que son identificados por su **DNI**, tienen su **nombre**, **apellidos**, **email** y **teléfono**. Los precios de reparación se calculan en coste de las piezas a reparar más horas de trabajo realizadas. Un coche (turismo) se repara a 20€ la hora, una motocicleta a 10€ la hora y los furgones a 25€ la hora, con un sobrecoste sobre el total del coste de las piezas del 10%. Cuando un cliente llega a nuestro taller, nos entrega el vehículo, que es diagnosticado por uno de nuestros mecánicos, se le asigna un código alfanumérico de trabajo formado por "AñoMesDíaMatricula", por ejemplo, si nos lo traen el 4 de abril del 2025 y la matrícula del vehículo es la "1435GTF" el código de trabajo es: "202504041435GTF". Una vez determinado el problema y asignado el código de trabajo se almacena la descripción del trabajo a realizar, el coste de las piezas, y las horas de trabajo a realizar y el DNI del mecánico que realizará la reparación, además de un valor que determina si el trabajo se ha terminado y otro para determinar si se ha cobrado (y por lo tanto el cliente se ha llevado su vehículo). Los mecánicos de nuestros talleres tienen su DNI, nombre, apellidos, email, teléfono y año de llegada a la empresa.

Ejercicio 1: Realiza el modelo del problema planteado, con las clases: Persona, Cliente, Mecánico, Vehículo, Motocicleta, Turismo, Furgón, TrabajoTaller y Taller. Usa herencia tantas veces como sea posible. Usa Mapas para la gestión de los clientes y para la gestión de los trabajos de taller, listas para cualquier otra necesidad de agregar/componer. (4p) (0,375 por clase, excepto Taller: 1p)

Ejercicio 2: Programa dos clases diferentes, una que implemente el interfaz Comparator<T> para la clase TrabajoTaller (ordenado por código) y otra para la Clase Persona (que ordene por Apellidos+Nombre). NO SON CLASES ANÓNIMAS (2p, un punto por clase)

Ejercicio 3: Realiza una clase Controlador que implemente el Patrón de Diseño Singleton y nos sirva de intermediario entre la vista (Main de formato texto) y el modelo. Tendrá como mínimo una instancia de la clase Taller para almacenar nuestro modelo. (1p, completo)

Ejercicio 4: Realiza las siguientes opciones: (3p) (0,375 por apartado)

1. Asignar CIF y descripción al taller
2. Dar de alta un cliente
3. Dar de alta un mecánico
4. Dar de alta un vehículo
5. Asignar trabajo a un vehículo
6. Listar los clientes (ordenados por Apellidos+Nombre)
7. Listar los Trabajos del Taller por hacer (ordenados por código)
8. Listar los Trabajos del Taller hechos sin cobrar (ordenados por código)

Evaluación:

Nota para superar el examen: 5.

PARTE 2: Interfaces gráficas de usuario (GUI)

Realiza el diseño en XML de una activity que nos permita recoger dos textos (con sus TextView y sus EditText) y que tenga un botón que cuando se pulsa construya un texto y lo asigne a otro TextView siguiendo las siguientes reglas:

- El primer TextView se llamará tvTextoUsuario y tendrá como texto "Usuario". Se ajustará al contenido tanto en anchura como en altura.
- El segundo TextView se llamará tvTextoServidor y tendrá como texto "Servidor". Se ajustará al contenido tanto en anchura como en altura.
- El tercer TextView se llamará tvResultado y tendrá como texto inicial "" (vacío). Se ajustará al contenido en altura y ocupará todo el ancho posible.
- El primer EditText se llamará etUser. Se ajustará al contenido en altura y ocupará todo el ancho posible.
- El segundo EditText se llamará etServer. Se ajustará al contenido en altura y ocupará todo el ancho posible.
- El botón se llamará btWork, y tendrá como texto "Concatenar". Se ajustará al contenido en altura y ocupará todo el ancho posible.
- El tercer TextView se llamará tvResult y estará inicialmente vacío. Se ajustará al contenido en altura y ocupará todo el ancho posible.
- Con respecto al orden de los elementos en la pantalla, estarán de arriba a abajo en el siguiente orden: tvTextoUsuario, etUser, tvTextoServidor, etServer, btWork y tvResult.

Cuando se pulse el botón, queremos que se obtenga el texto de etUser y se concatene al texto de etServer pero poniendo una "@" en medio. Es decir, si etUser es "root" y etServer es "202.23.12.8" cuando se pulsa el botón se debe de poner en tvResult el texto "root@202.23.12.8"